

Detailed Report: Lensboy-Based Multicamera Calibration

Dataset: dark_frames-no_common_pose_frame | Reference cameras: cam5 and cam6 | Prepared for
Nila Maitra Chaity

Multicamera calibration workflow

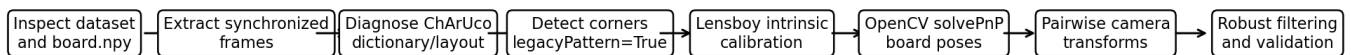


Figure 1. End-to-end workflow used for the multicamera calibration experiment.

Executive summary. This report documents the full multicamera calibration experiment performed after the previous single-camera Lensboy work. The goal was to process a seven-camera dark-frame dataset, obtain per-camera intrinsic models with Lensboy, and recover relative camera poses using pairwise board observations. The initial ChArUco detection failed, but diagnostics showed that the board required OpenCV legacy ChArUco layout mode. After enabling legacy mode, valid detections were recovered for all seven cameras. Each camera was then calibrated individually with Lensboy, board poses were estimated with OpenCV solvePnP, and a robust pairwise camera graph was estimated using cam5 and validated again with cam6.

1. Purpose and Context

The work described here extends an earlier successful single-camera Lensboy calibration pipeline to a multicamera dataset. In the earlier work, Lensboy was used to calibrate a single camera, evaluate reprojection error, remove bad frames, and generate diagnostic plots and an undistorted output video. The new task was harder because the dataset contains seven synchronized cameras and the image sequence is dark, with no frame where all cameras have a common valid board pose.

The main objective was to answer three questions:

Question	Meaning in this experiment
Can the ChArUco board be detected in all camera views?	Needed before any calibration can happen.
Can each camera be calibrated individually?	Needed to estimate the camera matrix and distortion for every camera.
Can the cameras be placed relative to each other?	Needed for multicamera rig calibration.

The final result is a robust relative camera graph. It is not a direct all-camera calibration from one shared board pose because the dataset contains zero frames where all seven cameras simultaneously detect the board. Instead, the rig was recovered pairwise through overlapping camera pairs.

2. Dataset and Board Configuration

The dataset used for this experiment is **dark_frames-no_common_pose_frame**. It contains seven camera videos, named cam1 to cam7. Each video has 475 frames, recorded at 10 FPS, with a duration of 47.5 seconds. Cameras cam1 to cam4 have resolution 2448 x 2048, while cam5 to cam7 have resolution 1440 x 1088.

Property	Value
Dataset name	dark_frames-no_common_pose_frame
Number of cameras	7
Frames per camera	475
Frame rate	10 FPS
Duration	47.5 seconds
Resolution cam1-cam4	2448 x 2048
Resolution cam5-cam7	1440 x 1088

The board description was read from the board numpy file. The values indicated a 6 x 8 ChArUco board with square_size_real = 3, marker_size = 0.6 relative to the square size, and dictionary_type = 0. Therefore, the physical marker size was interpreted as 0.6 x 3.0 = 1.8, and dictionary_type 0 was interpreted as DICT_4X4_50.

Board parameter	Value used
boardWidth	6
boardHeight	8
square_size_real	3.0
marker_size_real	1.8
dictionary	DICT_4X4_50
OpenCV legacy ChArUco layout	Enabled

3. Frame Extraction and Initial Detection

All 475 frames were extracted from each of the seven videos. This produced synchronized frame folders for cam1 through cam7. Keeping all frames was important because the frame index is used as the time correspondence between cameras. For example, frame_000100.jpg from cam5 and frame_000100.jpg from cam6 represent the same time instant.

The first ChArUco detection attempt used the board size 6 x 8, marker size 1.8, dictionary DICT_4X4_50, and a minimum validity threshold of 8 ChArUco corners. This initial attempt produced zero valid frames for all seven cameras. At that point, calibration could not continue, so a diagnostic step was required.

4. Dictionary and Board-Layout Diagnostics

The first diagnostic checked whether the ArUco dictionary was wrong. Several dictionaries were tested. The 4x4 family produced marker detections, while 5x5 and larger dictionaries produced almost nothing. This showed that the marker family was correct and the problem was not simply a wrong dictionary.

The second diagnostic tested ChArUco board-layout variants: 6 x 8 versus 8 x 6, marker size 1.8 versus 0.6, and OpenCV legacy layout mode off versus on. This was the key step. The correct configuration was:

```
DICT_4X4_50, 6 x 8 board, marker size 1.8, legacyPattern = True
```

Without legacy mode, OpenCV could detect some ArUco markers but could not correctly interpolate ChArUco corners. After enabling legacy mode with board.setLegacyPattern(True), the board became interpretable and detection quality improved dramatically.

5. Final ChArUco Detection Result

With legacy ChArUco layout enabled, detection succeeded for all cameras. The result is summarized below.

Camera	Valid frames	Total frames	Valid percentage
cam1	51	475	10.7%
cam2	56	475	11.8%
cam3	67	475	14.1%
cam4	65	475	13.7%
cam5	204	475	42.9%
cam6	153	475	32.2%
cam7	118	475	24.8%

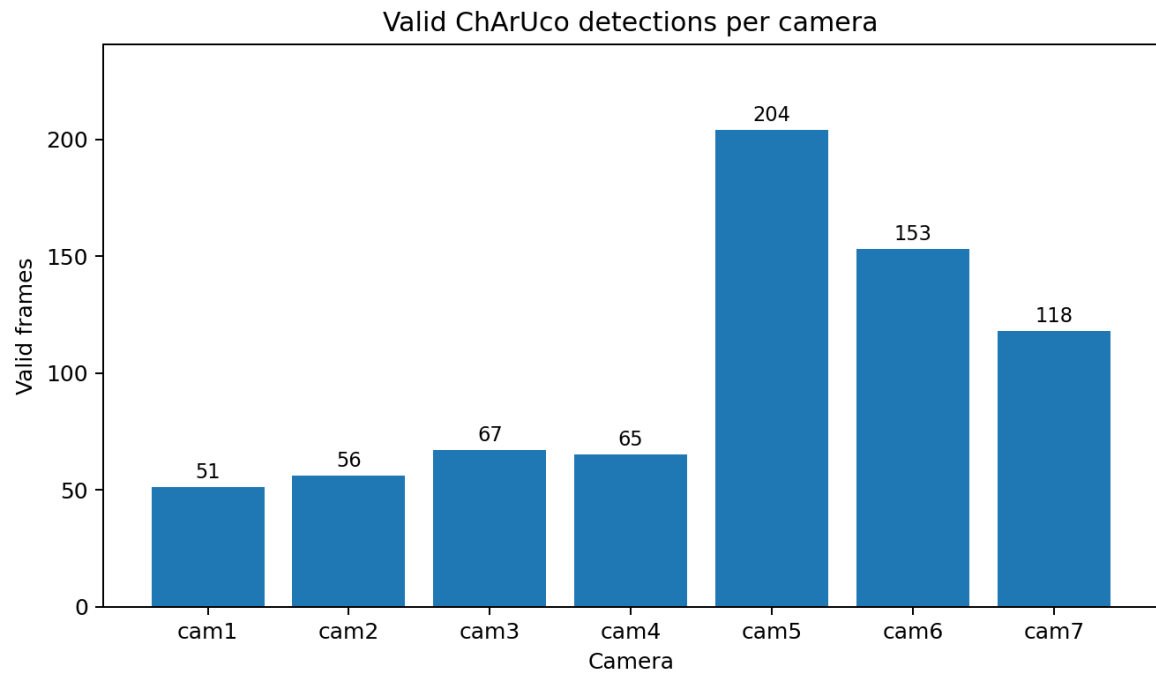


Figure 2. Number of valid ChArUco detections per camera after enabling legacy layout mode.

The result shows that cam5, cam6, and cam7 had the strongest detections. Cam1 to cam4 had fewer detections, but still enough valid frames for individual camera calibration.

6. Where Lensboy Was Used

Lensboy was used specifically for the per-camera intrinsic calibration stage. OpenCV was used for ChArUco detection and later for solvePnP board-pose estimation. The multicamera relative pose graph was computed with custom Python and NumPy logic.

For every camera, the detected ChArUco corner IDs and their 2D image coordinates were converted into Lensboy Frame objects. Lensboy then received the known 3D ChArUco target points and the observed 2D corner locations from all valid frames. The calibration call used the OpenCV camera model configuration:

```
result = lb.calibrate_camera(target_points, frames,
                             camera_model_config=lb.OpenCVConfig(image_height=image_height, image_width=image_width))
```

The Lensboy model estimated each camera matrix K and distortion coefficients. This is the single-camera intrinsic model. It explains how a 3D point on the board projects into the 2D image and how lens distortion changes that projection.

Lensboy output	Purpose
Camera matrix K	Contains fx, fy, cx, cy. These describe focal length and principal point.
Distortion coefficients	Describe radial and tangential lens distortion.
Residual plot	Shows how far detected points are from the projected points after calibration.
Detection coverage plot	Shows where on the image plane detections occur.
Inlier coverage plot	Shows which detections remain useful after model fitting.
Per-image RMS plot	Shows which frames have high reprojection error.

Lensboy did not directly solve the seven-camera rig. Instead, it produced reliable intrinsics for each camera. Those intrinsics were then used in the multicamera stages.

7. Per-Camera Intrinsic Calibration

All seven cameras were successfully calibrated with Lensboy using the valid ChArUco detections. The number of usable frames per camera matched the detection counts. Lensboy also generated per-camera diagnostic plots in each camera folder, including residuals, detection coverage, inlier coverage, and per-image RMS.

Camera	Lensboy calibration status	Usable frames
cam1	Success	51
cam2	Success	56
cam3	Success	67
cam4	Success	65
cam5	Success	204
cam6	Success	153
cam7	Success	118

8. Board Pose Estimation with solvePnP

After the intrinsic calibration, the next step was to estimate the pose of the ChArUco board in every valid frame for every camera. For each valid camera-frame pair, OpenCV solvePnP was used to estimate the transform from board coordinates to camera coordinates:

```
T_cam_board = pose of the board in the camera coordinate system
```

This pose estimation uses the Lensboy-calibrated camera matrix and distortion coefficients. Therefore, Lensboy is still important in this step, because solvePnP depends on the intrinsics estimated by Lensboy.

Camera	Poses estimated	Mean RMS px	Median RMS px	Max RMS px
cam1	51	2.815	1.621	10.327
cam2	56	1.065	0.912	3.576
cam3	67	1.455	0.825	7.067
cam4	65	3.443	2.167	24.480
cam5	204	0.188	0.157	1.000
cam6	153	0.590	0.549	1.747
cam7	118	1.278	1.162	3.563

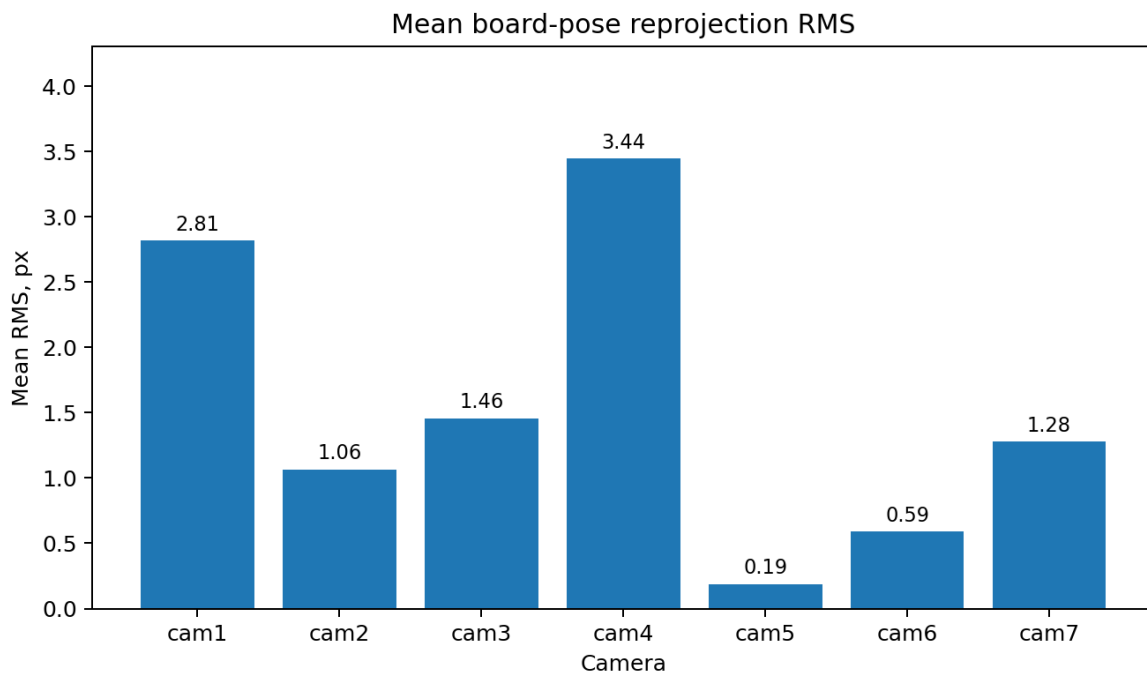


Figure 3. Mean board-pose reprojection RMS after solvePnP.

The best board-pose quality was obtained for cam5 and cam6. Cam4 had the highest maximum pose RMS, which means some individual board poses were less stable. This was handled later by RMS filtering and rotation-outlier filtering.

9. Pairwise Common Frames and the No-Common-Pose Limitation

The dataset name already suggested that there might be no common board pose frame across all cameras. This was confirmed: there were zero frames where all seven cameras simultaneously had a valid board pose. Therefore, a direct all-camera calibration from common frames was not possible.

However, many camera pairs shared valid board poses. The strongest overlaps were cam5-cam6, cam5-cam7, and cam6-cam7. This makes cam5 and cam6 good reference candidates for a pairwise camera graph.

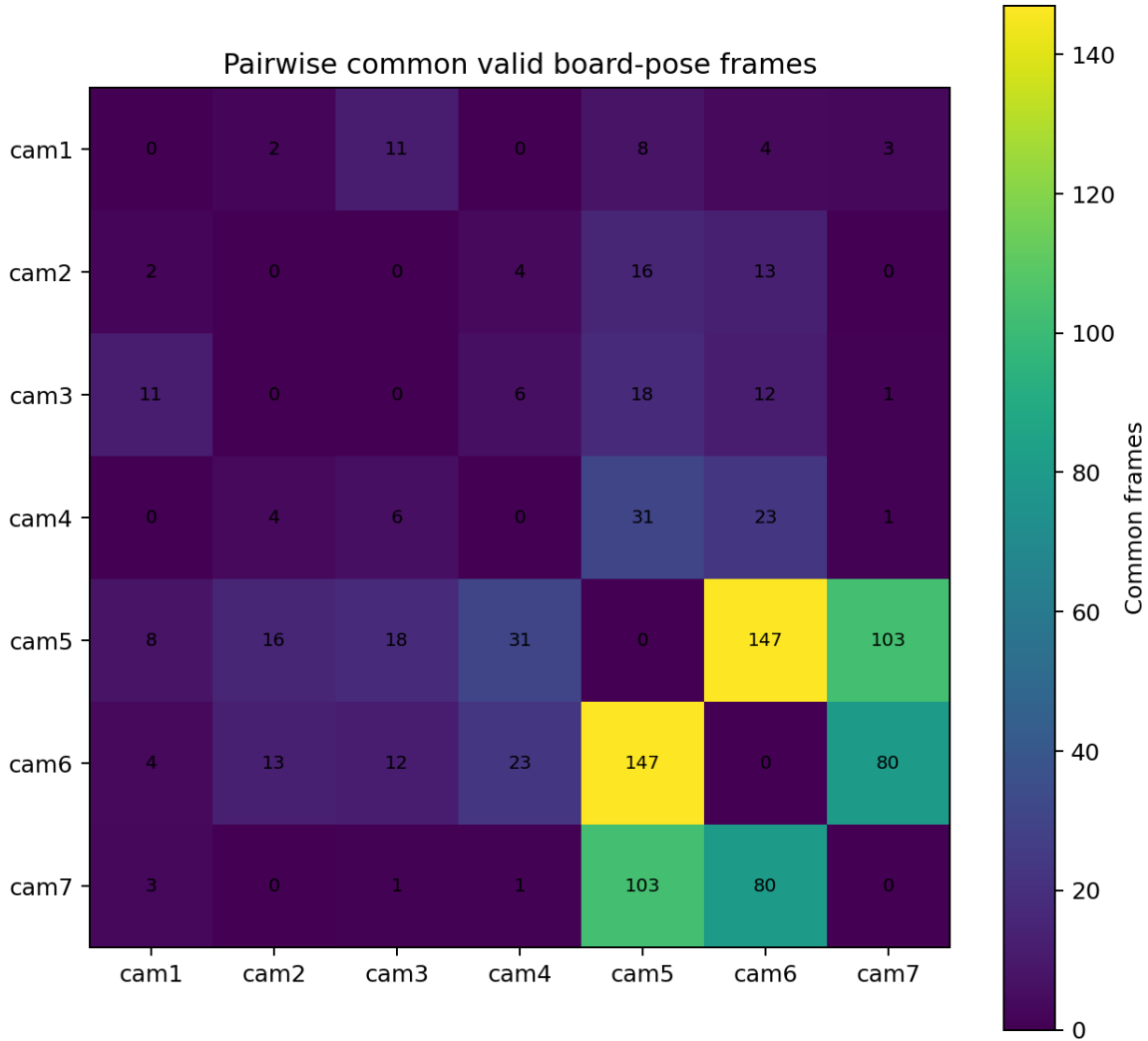


Figure 4. Pairwise common valid board-pose frames. The strongest overlaps are among cam5, cam6, and cam7.

The absence of a global common frame changed the strategy. Instead of using one frame seen by all cameras, relative transforms were estimated pairwise and connected through a reference camera.

10. Relative Camera Pose Estimation

For each common frame between a reference camera and another camera, both cameras observed the same physical board. For example, if cam5 and cam6 both have a board pose in the same frame, we know:

```
T_cam5_board and T_cam6_board
```

From these two board poses, the relative transform from cam6 to cam5 can be computed as:

```
T_cam5_cam6 = T_cam5_board x inverse(T_cam6_board)
```

This was repeated for every common frame. Each common frame produced one candidate relative transform. The candidate transforms were then averaged robustly to produce one final relative camera transform for each camera pair.

11. Robust Filtering Strategy

Two filtering stages were used to make the relative poses more reliable.

Filter	Threshold	Purpose
Pose RMS filter	2 px	Removes board poses with high reprojection error before relative transforms are estimated
Rotation outlier filter	5 deg	Removes candidate relative transforms that disagree strongly with the average transform

This robust filtering was especially important for cam7. Before rotation-outlier filtering, the cam5-cam7 link had a maximum rotation inconsistency of about 34.5 deg. After removing two rotation outliers, the maximum rotation inconsistency dropped to about 4.12 deg.

12. Robust Relative Calibration Using cam5 as Reference

cam5 was selected as the main reference camera because it had the best board-pose quality and strong pairwise overlap with several other cameras. The final robust settings were: reference camera cam5, pose RMS threshold 2 px, and rotation outlier threshold 5 deg.

Link	Common frames	Candidates used	Outliers removed	Median rot deg	Max rot deg	Median trans consistency
cam5 <- cam1	8	5	0	0.364	0.636	0.078
cam5 <- cam2	16	16	0	0.353	1.502	0.147
cam5 <- cam3	18	12	0	0.267	0.993	0.187
cam5 <- cam4	31	24	0	0.241	1.845	0.146
cam5 <- cam6	147	147	0	0.277	4.047	0.338
cam5 <- cam7	103	97	2	0.560	4.120	0.502

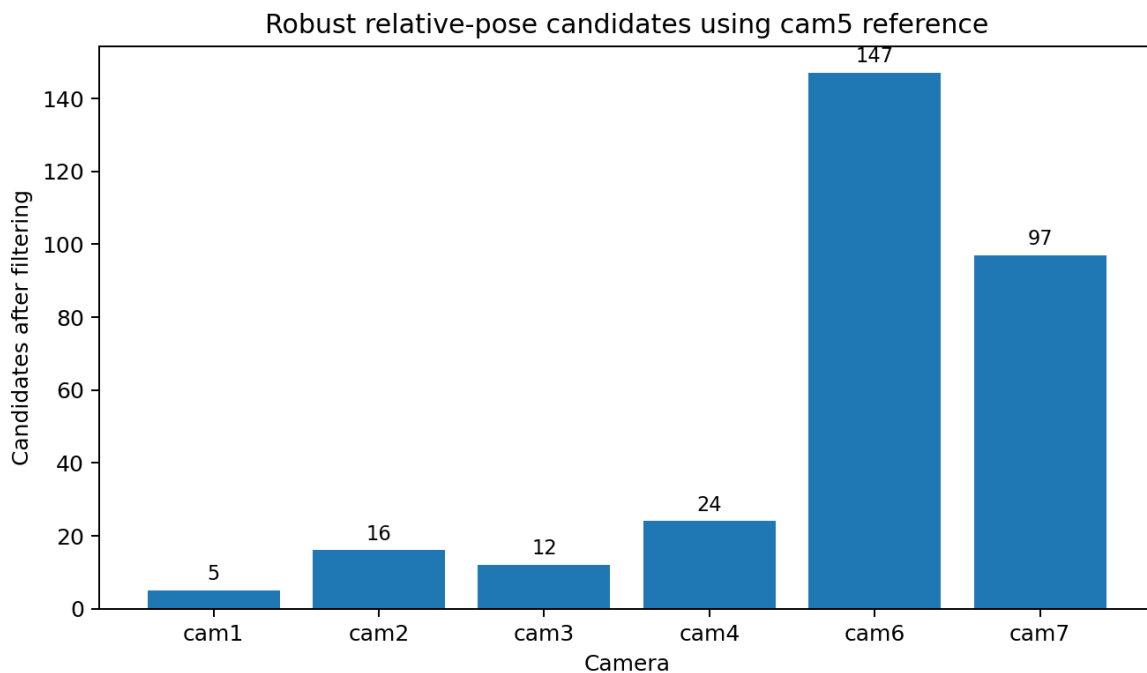


Figure 5. Candidate relative transforms after robust filtering using cam5 as reference.

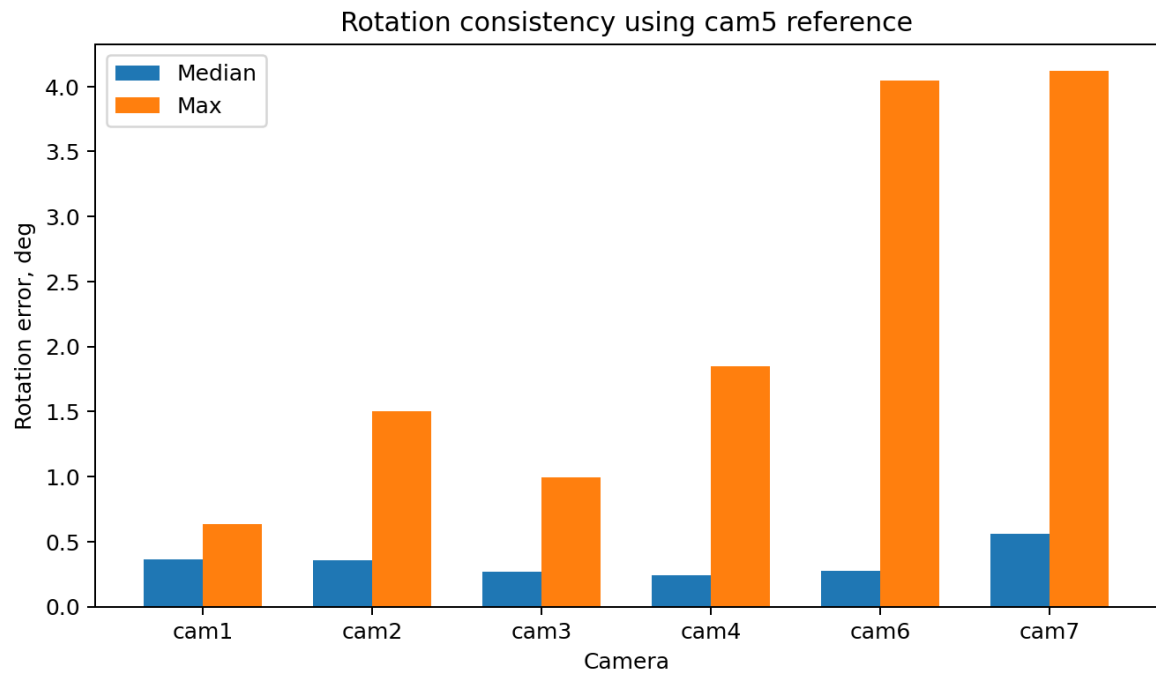


Figure 6. Median and maximum rotation consistency using cam5 as reference.

13. Camera Rig Visualization

The final robust camera rig was visualized in cam5 coordinates. In this plot, cam5 is placed at the origin and all other camera positions are expressed relative to cam5. The plot confirms that the rig graph is connected through cam5.

Robust estimated camera rig in cam5 coordinates

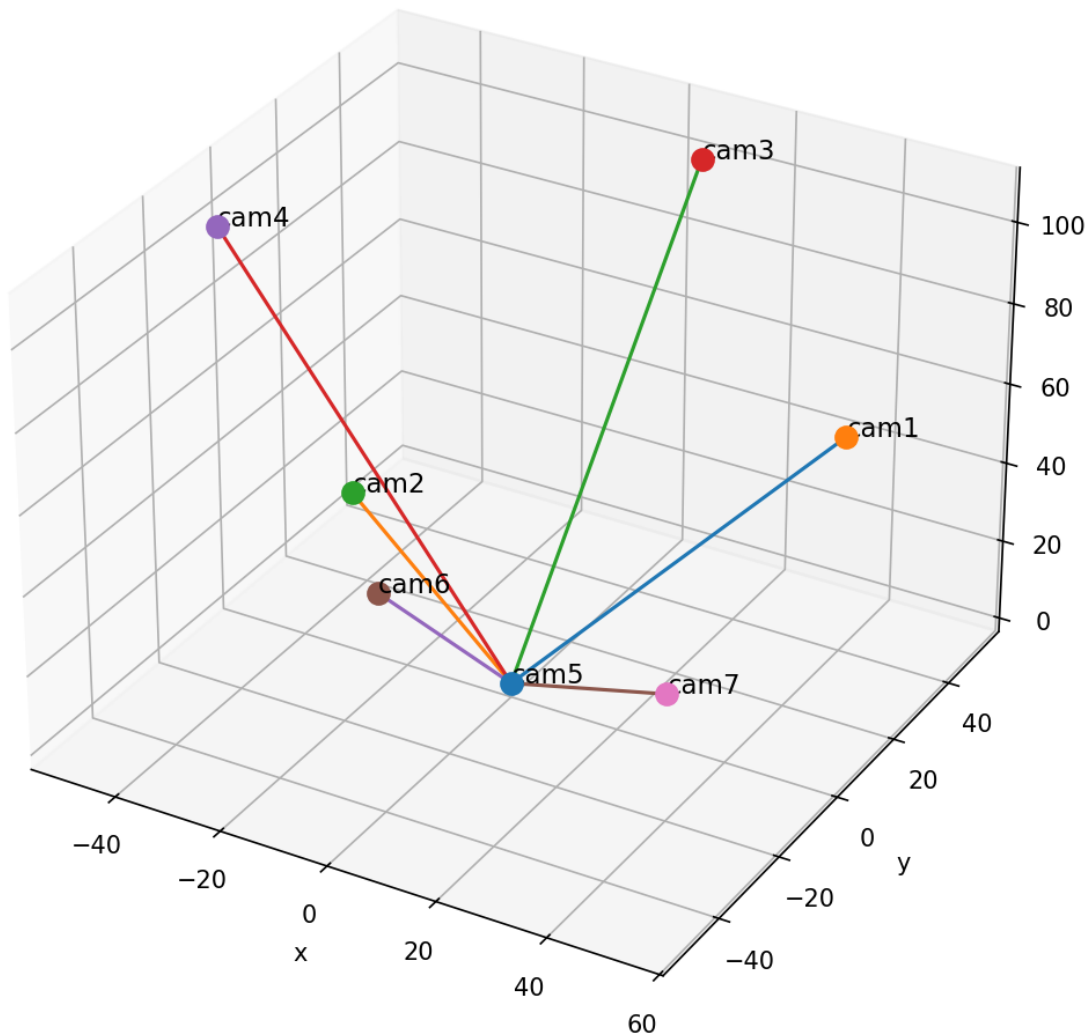


Figure 7. Robust estimated camera rig in cam5 coordinates.

Important interpretation note: the axes in this plot are the coordinate axes of cam5, not real-world room axes. Therefore, the z-axis should be interpreted as camera-coordinate depth, not physical height. The purpose of the plot is to verify that the relative camera graph is connected and geometrically plausible.

14. Validation Using cam6 as a Second Reference

To check whether the result was dependent only on the choice of cam5, the robust relative calibration was repeated using cam6 as reference. The cam6-reference result was consistent, especially for the strong cam6-cam5 and cam6-cam7 links.

Link	Common frames	Candidates used	Outliers removed	Median rot deg	Max rot deg	Median trans consistency
cam6 <- cam1	4	4	0	0.425	0.602	0.308
cam6 <- cam2	13	13	0	0.302	0.534	0.124
cam6 <- cam3	12	9	0	0.298	1.298	0.289

Link	Common frames	Candidates used	Outliers removed	Median rot deg	Max rot deg	Median trans consistency
cam6 <- cam4	23	22	0	0.223	2.159	0.172
cam6 <- cam5	147	147	0	0.277	4.047	0.375
cam6 <- cam7	80	75	2	0.648	4.038	0.624

The consistency under a second reference camera indicates that the strongest part of the rig, especially cam5-cam6-cam7, is stable and not an artifact of one arbitrary reference choice.

15. Quality Interpretation

The calibration result can be interpreted as follows:

Camera or link	Confidence	Reason
cam5-cam6	Strong	147 robust candidates and stable rotation consistency.
cam5-cam7	Strong after filtering	97 robust candidates; two rotation outliers removed.
cam6-cam7	Strong after filtering	75 robust candidates in cam6-reference validation.
cam5-cam4	Good	24 robust candidates and low median rotation error.
cam5-cam2	Moderate	16 robust candidates and low rotation error.
cam5-cam3	Moderate	12 robust candidates and low rotation error.
cam5-cam1	Low confidence but usable	Only 5 robust candidates, although rotation consistency is good.

The strongest part of the rig is the cam5-cam6-cam7 group. cam1 is the weakest camera because it has only a few common valid frames with the reference cameras. The result should therefore be described as a robust pairwise camera graph, not as a dense all-camera calibration from simultaneous common observations.

16. Final Conclusion

The multicamera calibration experiment was successful after a key diagnostic correction. The initial ChArUco detection failed because OpenCV was using the non-legacy board layout. Once OpenCV legacy ChArUco layout mode was enabled, all cameras produced valid detections. Lensboy was then used to calibrate each camera individually and generate single-camera intrinsic models. These models were used with OpenCV solvePnP to estimate board poses. Since no frame was common across all seven cameras, the multicamera rig was recovered using pairwise relative camera transforms. Robust filtering removed poor pose candidates and rotation outliers, producing a stable camera graph using both cam5 and cam6 as reference cameras.

In short: Lensboy provided the calibrated camera models, OpenCV provided board detection and pose estimation, and custom pairwise pose-graph logic produced the multicamera relative calibration.

17. Output Files Produced

Output type	Location or naming pattern
Per-camera Lensboy models	outputs/multicamera/dark_frames-no_common_pose_frame/camX/models/lensboy_opencv_model.json
Lensboy plots	outputs/multicamera/dark_frames-no_common_pose_frame/camX/plots/lensboy_*.png
Board poses	outputs/multicamera/dark_frames-no_common_pose_frame/board_poses/camX/board_poses.json
Common pose summary	outputs/multicamera/dark_frames-no_common_pose_frame/reports/dark_multicam_common_pose_frames.json
Robust relative poses, cam5 reference	outputs/multicamera/dark_frames-no_common_pose_frame/final/final_robust_relative_camera_poses_reference_cam5.json
Robust relative poses, cam6 reference	outputs/multicamera/dark_frames-no_common_pose_frame/final/final_robust_relative_camera_poses_reference_cam6.json
Robust rig plot	outputs/multicamera/dark_frames-no_common_pose_frame/plots/robust_camera_rig_reference_cam5.png

18. Recommended Next Steps

The current result is strong enough for a progress report. For future improvement, the next steps would be: collect or identify frames with more common camera overlap, add global bundle adjustment over all pairwise transforms, inspect Lensboy residual plots for each camera, and validate the recovered rig against any known physical camera layout if available.